

Spring Boot

Spring Boot is Spring with the configuration already written. Auto-configuration is the part that writes it: it looks at what is on the classpath, guesses what we want, and creates those beans for us.

The rule that makes it safe is the second half. **Auto-configuration creates nothing that we have already configured ourselves.**

Contents

1. [Inversion of Control and Dependency Injection](#)
2. [Declaring beans](#)
3. [The bean lifecycle](#)
4. [The problem it solves](#)
5. [@SpringBootApplication](#)
6. [How it works](#)
7. [The conditions](#)
8. [Starters](#)
9. [Overriding it](#)
10. [Seeing what happened](#)
11. [Writing our own](#)

1. Inversion of Control and Dependency Injection

IoC (Inversion of Control) means the framework creates our objects and connects them together, instead of each class doing it with `new`.

DI (Dependency Injection) is how that happens: a class declares what it needs, and the container supplies it.

Without it, a class chooses its own dependency:

```
public class OrderService {  
  
    private final PaymentClient paymentClient =  
        new PaymentClient("https://pay.example.com");  
}
```

That class is now tied to one implementation and one URL. A test cannot replace the client, because nothing outside the class can reach it.

With it, the class only states what it needs:

```

@Service
public class OrderService {

    private final PaymentClient paymentClient;

    public OrderService(PaymentClient paymentClient) {
        this.paymentClient = paymentClient;
    }
}

```

The control over which `PaymentClient` is used moved out of the class. That move is the inversion.

A test can now supply anything that fits:

```

OrderService service = new OrderService(new FakePaymentClient());

```

The container is `ApplicationContext`. It reads the bean definitions, creates the objects, works out what each one needs, and supplies it. A bean is simply an object the container created and manages.

There are three ways to inject a dependency:

Way	Written as	Use it
Constructor	A constructor parameter	Almost always
Setter	A <code>setX</code> method with <code>@Autowired</code>	For a genuinely optional dependency
Field	<code>@Autowired</code> on the field itself	Avoid

```

// Constructor: the dependency is required and the field can be final
public OrderService(PaymentClient paymentClient) {
    this.paymentClient = paymentClient;
}

// Setter: the bean works without it
@Autowired(required = false)
public void setAuditClient(AuditClient auditClient) {
    this.auditClient = auditClient;
}

// Field: shortest to write, and the weakest
@Autowired
private PaymentClient paymentClient;

```

Field injection looks tidy and costs three things: the field cannot be `final`, the dependencies are invisible from outside the class, and a plain unit test has to use reflection to set them.

Since Spring 4.3 a class with one constructor needs no `@Autowired` on it.

2. Declaring beans

There are two ways to tell the container about a bean.

Annotate the class, and let the component scan find it:

```
@Component    // anything
@Service       // business logic
@Repository    // data access, and it translates database exceptions
@Controller    // web endpoints
```

All four are `@Component` underneath. The names carry intent, and two of them add behavior on top.

Declare it in a `@Configuration` class, which is the only option for a class we did not write and cannot annotate:

```
@Configuration
public class ClientConfig {

    @Bean
    public PaymentClient paymentClient(PaymentProperties properties) {
        return new PaymentClient(properties.getUrl());
    }
}
```

A bean has a scope, which decides how many instances exist:

Scope	How many
<code>singleton</code>	One for the whole container. The default
<code>prototype</code>	A new one every time it is asked for
<code>request</code>	One per HTTP request
<code>session</code>	One per HTTP session

```
@Bean
@Scope("prototype")
public ReportBuilder reportBuilder() {
    return new ReportBuilder();
}
```

Singleton being the default has a consequence worth stating. One instance is shared by every caller and every thread, so a singleton bean must not keep per-request state in a field.

When two beans of the same type exist, the container needs to be told which one:

```
@Bean
@Primary
public PaymentClient cardClient() { return new CardClient(); }

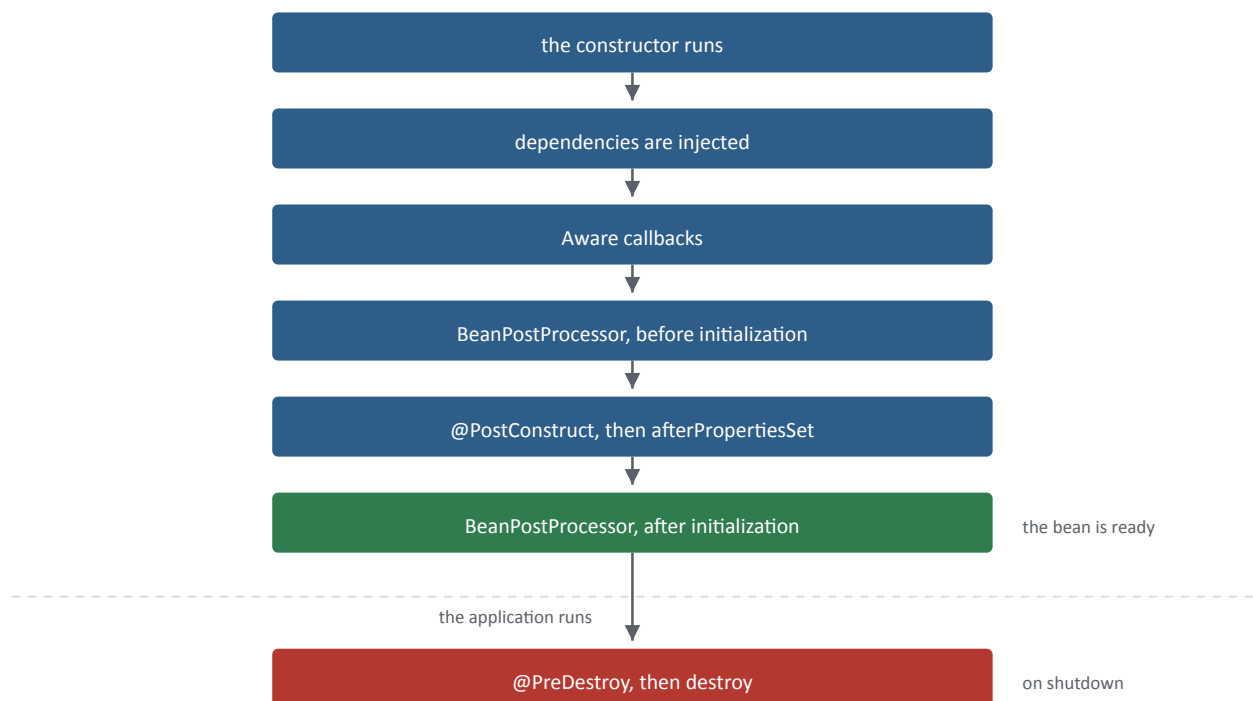
@Bean
public PaymentClient upiClient() { return new UpiClient(); }
```

`@Primary` picks the default. `@Qualifier` overrides it at one injection point:

```
public OrderService(@Qualifier("upiClient") PaymentClient paymentClient) {
    this.paymentClient = paymentClient;
}
```

3. The bean lifecycle

Every bean goes through the same steps between the container starting and stopping.



Constructor injection is the only kind finished by the first step. Setter and field injection complete at the second.

A bean that uses the two common hooks:

```

@Component
public class CacheWarmer {

    private final ProductRepository repository;

    public CacheWarmer(ProductRepository repository) {
        this.repository = repository;
        System.out.println("1. constructor");
    }

    @PostConstruct
    public void warmUp() {
        System.out.println("2. post construct, repository is ready");
    }

    @PreDestroy
    public void flush() {
        System.out.println("3. pre destroy");
    }
}

```

Output, with the last line printed when the application stops:

1. constructor
2. post construct, repository is ready
3. pre destroy

Hook	When it runs	Use it
Constructor	First	For required dependencies
<code>@PostConstruct</code>	After injection, before the bean is used	For startup work
<code>afterPropertiesSet</code>	The same moment	No, it ties the class to Spring
<code>@Bean(initMethod =)</code>	The same moment	For a class we cannot annotate
<code>@PreDestroy</code>	On a graceful shutdown	For releasing something
<code>destroy</code>	The same moment	No, it ties the class to Spring

Two points interviews ask about.

A field-injected dependency is null inside the constructor. The constructor runs before injection, so anything needed there has to be a constructor parameter. This is one more reason constructor injection is the default answer.

`@PreDestroy` never runs on a prototype bean. The container creates it, hands it over, and stops tracking it, so it has nothing to call back on.

4. The problem it solves

Plain Spring needs every bean declared. A database and nothing else looks like this:

```
@Configuration
public class DataConfig {

    @Bean
    public DataSource dataSource() {
        HikariDataSource ds = new HikariDataSource();
        ds.setJdbcUrl("jdbc:postgresql://localhost:5432/payroll");
        ds.setUsername("app");
        ds.setPassword("secret");
        return ds;
    }

    @Bean
    public JdbcTemplate jdbcTemplate(DataSource dataSource) {
        return new JdbcTemplate(dataSource);
    }

    @Bean
    public PlatformTransactionManager transactionManager(DataSource dataSource) {
        return new DataSourceTransactionManager(dataSource);
    }
}
```

Spring Boot replaces all of it with three properties:

```
spring.datasource.url=jdbc:postgresql://localhost:5432/payroll
spring.datasource.username=app
spring.datasource.password=secret
```

and one class:

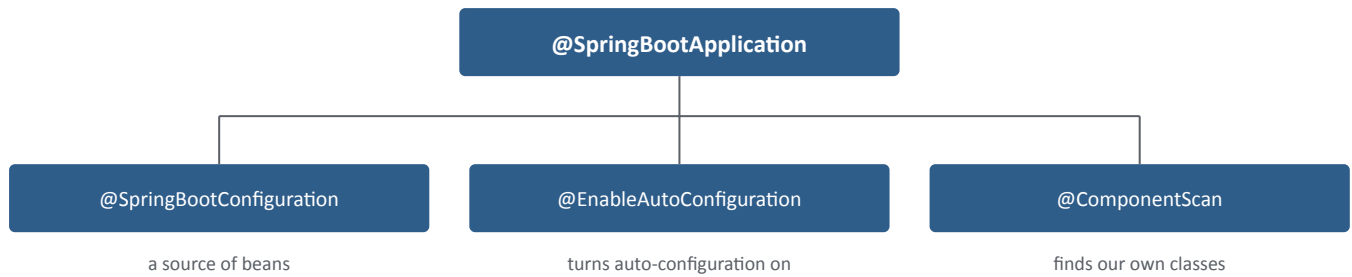
```
@SpringBootApplication
public class PayrollApplication {

    public static void main(String[] args) {
        SpringApplication.run(PayrollApplication.class, args);
    }
}
```

The beans are the same. Auto-configuration is what creates them.

5. @SpringBootApplication

That one annotation is three annotations together.



Only the middle one does auto-configuration. The other two are ordinary Spring.

```
@SpringBootApplication
public class PayrollApplication { }

// means the same as

@SpringBootConfiguration
@EnableAutoConfiguration
@ComponentScan
public class PayrollApplication { }
```

`@ComponentScan` starts from the package this class sits in and searches downward. That is why the main class belongs at the root of the package tree: a class in a sibling package is never found.

6. How it works

At startup, in this order:



Our beans are registered before auto-configuration is evaluated. That ordering is what lets it skip what we already defined.

The list of auto-configuration classes is not found by scanning. Every jar that supplies one carries a plain text file:

```
META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports
```

Each line names a class:

```
org.springframework.boot.autoconfigure.jdbc.DataSourceAutoConfiguration
org.springframework.boot.autoconfigure.web.servlet.WebMvcAutoConfiguration
org.springframework.boot.autoconfigure.jackson.JacksonAutoConfiguration
```

Before Spring Boot 2.7 the same list lived in `META-INF/spring.factories` under an `EnableAutoConfiguration` key. Plenty of material still shows that form, so it is worth recognizing.

7. The conditions

Every auto-configuration class is wrapped in conditions. They are what turn a long list of candidates into the few that apply.

Annotation	The bean applies when
<code>@ConditionalOnClass</code>	A class is on the classpath
<code>@ConditionalOnMissingClass</code>	A class is not on the classpath
<code>@ConditionalOnBean</code>	A bean of that type already exists
<code>@ConditionalOnMissingBean</code>	No bean of that type exists

Annotation	The bean applies when
<code>@ConditionalOnProperty</code>	A property is set, or set to a given value
<code>@ConditionalOnResource</code>	A file is present
<code>@ConditionalOnWebApplication</code>	The application is a web application
<code>@ConditionalOnExpression</code>	An expression evaluates to true

A simplified version of the one that builds a `DataSource`:

```
@AutoConfiguration
@ConditionalOnClass(DataSource.class)
@ConditionalOnProperty(prefix = "spring.datasource", name = "url")
public class DataSourceAutoConfiguration {

    @Bean
    @ConditionalOnMissingBean
    public DataSource dataSource(DataSourceProperties properties) {
        return properties.initializeDataSourceBuilder().build();
    }
}
```

Read out loud: if `DataSource` is on the classpath, and `spring.datasource.url` is set, and no `DataSource` bean exists already, then create one.

`@ConditionalOnMissingBean` is the important one. It is the mechanism behind creating nothing we already have. Every bean Spring Boot makes for us carries it, which is why defining our own version is all it takes to replace one.

8. Starters

A starter is a dependency that brings a set of libraries, and with them the auto-configuration that knows what to do with those libraries.

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
```

Starter	What appears
<code>spring-boot-starter-web</code>	Embedded Tomcat, Spring MVC, Jackson
<code>spring-boot-starter-data-jpa</code>	Hibernate, a <code>DataSource</code> , repository support
<code>spring-boot-starter-security</code>	A filter chain, a default user, a login page
<code>spring-boot-starter-actuator</code>	Health, metrics and info endpoints
<code>spring-boot-starter-test</code>	JUnit 5, Mockito, AssertJ, Spring Test

Adding a starter changes nothing by itself. It changes the classpath, and the classpath is what the conditions read:

```

Starter    → puts classes on the classpath
Classpath → decides which conditions pass
Conditions → decide which beans exist

```

That chain is the whole idea. Removing a starter removes beans for the same reason.

9. Overriding it

Three ways, from the one used most to the one used least.

Define the bean ourselves. `@ConditionalOnMissingBean` sees it and does nothing.

```

@Configuration
public class DataConfig {

    @Bean
    public DataSource dataSource() {
        HikariDataSource ds = new HikariDataSource();
        ds.setJdbcUrl("jdbc:postgresql://localhost:5432/payroll");
        ds.setMaximumPoolSize(40);
        return ds;
    }
}

```

Set a property. Most of what we would want to change is already exposed.

```

server.port=9090
spring.datasource.hikari.maximum-pool-size=40
spring.jpa.hibernate.ddl-auto=validate

```

Exclude the auto-configuration. Useful when a starter arrives through another dependency and brings something we do not want.

```
@SpringBootApplication(exclude = DataSourceAutoConfiguration.class)
public class PayrollApplication { }
```

```
spring.autoconfigure.exclude=\
    org.springframework.boot.autoconfigure.jdbc.DataSourceAutoConfiguration
```

10. Seeing what happened

"Why is this bean not there" is the common question, and there is a direct answer for it.

```
java -jar payroll.jar --debug
```

That prints the condition evaluation report:

```
Positive matches:
-----
    DataSourceAutoConfiguration matched:
        - @ConditionalOnClass found required class 'javax.sql.DataSource'

Negative matches:
-----
    RabbitAutoConfiguration:
        Did not match:
            - @ConditionalOnClass did not find required class
              'com.rabbitmq.client.Channel'
```

The negative matches are the useful half. They name the exact condition that failed, which is usually a missing dependency or an unset property.

With actuator on the classpath the same report is available while the application runs:

```
GET /actuator/conditions
```

11. Writing our own

A shared library inside a company is the normal reason to write one.

```

@AutoConfiguration
@ConditionalOnClass(AuditClient.class)
@EnableConfigurationProperties(AuditProperties.class)
public class AuditAutoConfiguration {

    @Bean
    @ConditionalOnMissingBean
    @ConditionalOnProperty(prefix = "audit", name = "enabled", havingValue = "true")
    public AuditClient auditClient(AuditProperties properties) {
        return new AuditClient(properties.getUrl(), properties.getTimeout());
    }
}

```

The properties class turns `audit.*` settings into a typed object:

```

@ConfigurationProperties(prefix = "audit")
public class AuditProperties {

    private String url;
    private Duration timeout = Duration.ofSeconds(2);

    // getters and setters
}

```

Then register the class, in the library's own jar:

```

src/main/resources/META-INF/spring/
    org.springframework.boot.autoconfigure.AutoConfiguration.imports

```

```
com.example.audit.AuditAutoConfiguration
```

Three rules keep it well behaved:

Rule	Reason
Put <code>@ConditionalOnMissingBean</code> on every bean	An application can then replace it
Never use <code>@ComponentScan</code> in it	It would pull in beans the application did not ask for
Keep it in its own module	The application depends on it, not the other way round

The pieces together

Piece	What it does
<code>ApplicationContext</code>	The container that creates and holds the beans
Constructor injection	The class states what it needs, the container supplies it
<code>@Component</code> , <code>@Bean</code>	The two ways to declare a bean
<code>@PostConstruct</code>	Startup work, after the dependencies are in place
<code>@SpringBootApplication</code>	Configuration, auto-configuration and component scan together
<code>AutoConfiguration.imports</code>	The list of candidate auto-configuration classes
<code>@ConditionalOnClass</code>	Applies when a library is present
<code>@ConditionalOnMissingBean</code>	Applies when we have not defined it ourselves
<code>@ConfigurationProperties</code>	Turns properties into a typed object
<code>--debug</code>	Prints which conditions passed and which failed

For example, imagine a payroll application. Adding `spring-boot-starter-data-jpa` puts Hibernate and a connection pool on the classpath, so `DataSourceAutoConfiguration` and the JPA auto-configuration both apply and the application gets a `DataSource` , an `EntityManagerFactory` and a transaction manager without a line of configuration. Three properties point it at the real database. Later the pool needs tuning, which is one more property. Later still the application needs two databases, so it declares both `DataSource` beans itself, and the auto-configuration stops creating one.

The most useful thing at this point is not to memorize the conditions. Start an application with `--debug` , find one bean in the positive matches, and follow the condition it names back to the dependency that put it there.